

I want a controller router for YearlyAnalysis please refer the controller and router attached for the format currently yearlyanalysis shows the analytics from dummy mock data defined but now I want it to show analytics from the real table data

I checked your existing **monthly analysis controller + router** and the **YearlyAnalysis frontend**. Your yearly page currently reads from mock data (mockData.yearlyValueData, mockData.winRateTrendData, mockData.yearSummaryData) instead of backend API. You can build getYearlyAnalysis() almost exactly like getMonthlyAnalysis(), but aggregate by **Financial Year (Apr → Mar)** instead of month.

1. Add controller (analysisController.js)

```
import db from "../models/index.js";

const DomesticLeads = db.DomesticLeadsModel;
const BudgetaryQuotation = db.BudgetaryQuotationModel;
const LostForm = db.LostFormModel;

// Reuse your parseDate function
function parseDate(dateStr) {
  if (!dateStr) return null;

  try {
    if (dateStr.includes("-") && dateStr.split("-")[0].length === 4) {
      return new Date(dateStr);
    }

    if (dateStr.includes("/")) {
      const [day, month, year] = dateStr.split("/");
      return new Date(Number(year), Number(month) - 1, Number(day));
    }

    if (dateStr.includes("-")) {
      const [day, month, year] = dateStr.split("-");
      return new Date(Number(year), Number(month) - 1, Number(day));
    }

    return null;
  } catch {
    return null;
  }
}
```

```

function getFinancialYear(date) {
  const year = date.getFullYear();
  const month = date.getMonth(); // Jan=0

  if (month >= 3) {
    // Apr onwards
    return `FY ${String(year).slice(2)}-${String(year + 1).slice(2)}`;
  } else {
    return `FY ${String(year - 1).slice(2)}-${String(year).slice(2)}`;
  }
}

```

```

export const getYearlyAnalysis = async (req, res) => {
  try {
    const leads = await DomesticLeads.findAll({ raw: true });
    const bqs = await BudgetaryQuotation.findAll({ raw: true });
    const lostForms = await LostForm.findAll({ raw: true });

    const yearlyMap = {};

    // ----- LEADS -----
    leads.forEach(row => {
      const date = parseDate(row.tenderDated);
      if (!date || isNaN(date.getTime())) return;

      const fy = getFinancialYear(date);

      if (!yearlyMap[fy]) {
        yearlyMap[fy] = {
          fy,
          leads: 0,
          bqs: 0,
          orders: 0,
          value: 0,
          won: 0,
          lost: 0
        };
      }

      yearlyMap[fy].leads++;
    });
  }
}

```

```

const value = parseFloat(row.estimatedValueInCrWithoutGST || 0);
if (!isNaN(value)) {
  yearlyMap[fy].value += value;
}

const status = (row.wonLostParticipated || "")
  .toLowerCase()
  .trim();

if (status.includes("won") || status.includes("win")) {
  yearlyMap[fy].won++;
  yearlyMap[fy].orders++;
}

if (status.includes("lost")) {
  yearlyMap[fy].lost++;
}
});

// ----- BQ -----
bqs.forEach(row => {
  const date = parseDate(row.dateOfLetterSubmission);
  if (!date || isNaN(date.getTime())) return;

  const fy = getFinancialYear(date);

  if (!yearlyMap[fy]) {
    yearlyMap[fy] = {
      fy,
      leads: 0,
      bqs: 0,
      orders: 0,
      value: 0,
      won: 0,
      lost: 0
    };
  }

  yearlyMap[fy].bqs++;
});

```

```

// Lost forms (if year column exists use that instead)
const totalLostForms = lostForms.length;

const result = Object.values(yearlyMap)
  .sort((a, b) => a.fy.localeCompare(b.fy))
  .map(row => {
    const winRate =
      row.leads > 0
        ? ((row.orders / row.leads) * 100).toFixed(1) + "%"
        : "0%";

    return {
      ...row,
      value: Number(row.value.toFixed(2)),
      winRate
    };
  });

res.status(200).json({
  success: true,
  totalLostForms,
  data: result
});
} catch (error) {
  console.error("Yearly Analysis Error:", error);

  res.status(500).json({
    success: false,
    message: error.message
  });
}
};

```

2. Update router (analysisRoutes.js)

```

import express from "express";
import {
  getMonthlyAnalysis,
  getYearlyAnalysis
} from "../controllers/analysisController.js";

const router = express.Router();

```

```
router.get("/monthly-analysis", getMonthlyAnalysis);
router.get("/yearly-analysis", getYearlyAnalysis);
```

```
export default router;
```

3. Frontend API call (YearlyAnalysis.js)

Replace mock data usage with API fetch:

```
const [yearlyData, setYearlyData] = useState([]);
```

```
useEffect(() => {
  fetch(`${ServerIp}/api/yearly-analysis`)
    .then(res => res.json())
    .then(data => {
      if (data.success) {
        setYearlyData(data.data);
      }
    });
}, []);
```

Then replace:

Chart 1

```
<YearlyValueChart data={yearlyData} />
```

Chart 2

```
<WinRateTrendChart data={yearlyData} />
```

Table

Replace:

```
mockData.yearSummaryData.map(...)
```

with

```
yearlyData.map(...)
```

Important question (needed for exact controller)

I assumed:

- DomesticLeadsModel contains:
 - tenderDated
 - estimatedValueInCrWithoutGST
 - wonLostParticipated
- BudgetaryQuotationModel contains:
 - dateOfLetterSubmission
- LostFormModel only counted via row count

If your **YearlyAnalysis mock UI needs extra KPIs (Best Year, CAGR, Growth)**

calculated from DB too, send me:

1. YearlyValueChart component
2. mockData.js

Then I can generate the **complete ready-to-run backend + frontend integration code** for you.

budgetary_quotation_model(12).js

JavaScript

domestic_leads_model(12).js

JavaScript

lost_form_model(7).js

JavaScript

These are the models please refer them as well

Thanks — the models clarify a few important things. I spotted **3 field mismatches / improvements** in my previous controller suggestion.

Important model observations

Domestic Leads

Fields available:

- tenderDated
- estimatedValueInCrWithoutGST
- orderWonValueInCrWithoutGST ← better for actual order value
- wonLostParticipated

Budgetary Quotation

Field is:

estimateValueInCrWithoutGST

(not estimated...)

Lost Form

Good news — it has:

yearWeLost

So yearly lost counts can be grouped properly.

Updated getYearlyAnalysis (better version)

Use this instead of my earlier draft:

```
export const getYearlyAnalysis = async (req, res) => {
  try {
    const leads = await DomesticLeads.findAll({ raw: true });
    const bq = await BudgetaryQuotation.findAll({ raw: true });
    const lostForms = await LostForm.findAll({ raw: true });

    const yearlyMap = {};

    function getFinancialYear(date) {
      const year = date.getFullYear();
```

```

const month = date.getMonth(); // Jan=0

if (month >= 3) {
  return `FY ${String(year).slice(2)}-${String(year + 1).slice(2)}`;
} else {
  return `FY ${String(year - 1).slice(2)}-${String(year).slice(2)}`;
}
}

function ensureYear(fy) {
  if (!yearlyMap[fy]) {
    yearlyMap[fy] = {
      fy,
      leads: 0,
      bqs: 0,
      orders: 0,
      value: 0,
      won: 0,
      lost: 0
    };
  }
}

// ----- DOMESTIC LEADS -----
leads.forEach(row => {
  const date = parseDate(row.tenderDated);
  if (!date || isNaN(date)) return;

  const fy = getFinancialYear(date);
  ensureYear(fy);

  yearlyMap[fy].leads++;

  const leadValue = parseFloat(
    row.estimatedValueInCrWithoutGST || 0
  );

  if (!isNaN(leadValue)) {
    yearlyMap[fy].value += leadValue;
  }
}

```

```

const status = (row.wonLostParticipated || "")
  .toLowerCase()
  .trim();

if (status.includes("won") || status.includes("win")) {
  yearlyMap[fy].won++;
  yearlyMap[fy].orders++;

  // OPTIONAL:
  // if chart should show only won order value use this instead
  const orderValue = parseFloat(
    row.orderWonValueInCrWithoutGST || 0
  );

  if (!isNaN(orderValue) && orderValue > 0) {
    yearlyMap[fy].value += orderValue;
  }
}

if (status.includes("lost")) {
  yearlyMap[fy].lost++;
}
});

// ----- BQ -----
bqs.forEach(row => {
  const date = parseDate(row.dateOfLetterSubmission);
  if (!date || isNaN(date)) return;

  const fy = getFinancialYear(date);
  ensureYear(fy);

  yearlyMap[fy].bqs++;
});

// ----- LOST FORMS -----
lostForms.forEach(row => {
  const fy = row.yearWeLost;
  if (!fy) return;

  ensureYear(fy);

```



```

    yearlyMap[fy].lost++;
  });

  const sorted = Object.values(yearlyMap).sort((a, b) =>
    a.fy.localeCompare(b.fy)
  );

  const result = sorted.map((row, index) => {
    let growth = "Baseline";

    if (index > 0) {
      const prev = sorted[index - 1].value;

      if (prev > 0) {
        const percentage = ((row.value - prev) / prev) * 100;
        growth = `${percentage >= 0 ? "+" : ""}${percentage.toFixed(1)}%`;
      }
    }

    return {
      ...row,
      value: Number(row.value.toFixed(2)),
      winRate:
        row.leads > 0
          ? ((row.orders / row.leads) * 100).toFixed(1) + "%"
          : "0%",
      growth
    };
  });

  res.status(200).json({
    success: true,
    data: result
  });

} catch (error) {
  console.error("Yearly analysis error:", error);

  res.status(500).json({
    success: false,
    message: error.message
  });
}

```

```
});  
}  
};
```

One more thing (very important)

I still need to see **your mockData.js for yearly analytics**.

Because your frontend expects exactly:

yearlyValueData

winRateTrendData

yearSummaryData

yearlyKpis

Right now I can build backend aggregation, **but I don't know the exact response shape needed by your charts** (YearlyValueChart, WinRateTrendChart).

Please send either:

- mockData.js **OR**
- Charts.jsx

Then I can generate the **exact API response format + ready frontend integration** with zero guesswork.

mockData(2).js

JavaScript

Charts(7).js

JavaScript

Here you go

Perfect — now I can see exactly what your charts expect.

Required frontend data shape

From your chart components:

YearlyValueChart

```
[  
  { fy: "FY 21-22", value: 148 }  
]
```

WinRateTrendChart

```
[  
  { fy: "FY 21-22", rate: 27 }  
]
```

Summary Table

```
[  
  {  
    fy,  
    bqs,  
    leads,  
    orders,
```

```

    value,
    winRate,
    lost,
    growth
  }
]

```

That matches your mock data exactly.

Final backend response format

I recommend returning **all yearly page data in one API**:

```

{
  "success": true,
  "yearlyValueData": [],
  "winRateTrendData": [],
  "yearSummaryData": [],
  "yearlyKpis": []
}

```

Complete controller (ready to paste)

Add this inside analysisController.js

```

export const getYearlyAnalysis = async (req, res) => {
  try {
    const leads = await DomesticLeads.findAll({ raw: true });
    const bqs = await BudgetaryQuotation.findAll({ raw: true });
    const lostForms = await LostForm.findAll({ raw: true });

    const yearlyMap = {};

    function getFinancialYear(date) {
      const year = date.getFullYear();
      const month = date.getMonth();

      return month >= 3
        ? `FY ${String(year).slice(2)}-${String(year + 1).slice(2)}`
        : `FY ${String(year - 1).slice(2)}-${String(year).slice(2)}`;
    }

    function ensureYear(fy) {
      if (!yearlyMap[fy]) {
        yearlyMap[fy] = {
          fy,

```

```

    bqs: 0,
    leads: 0,
    orders: 0,
    value: 0,
    lost: 0
  };
}
}

// LEADS
leads.forEach(row => {
  const date = parseDate(row.tenderDated);
  if (!date || isNaN(date.getTime())) return;

  const fy = getFinancialYear(date);
  ensureYear(fy);

  yearlyMap[fy].leads++;

  const status = (row.wonLostParticipated || "")
    .toLowerCase()
    .trim();

  if (status.includes("won")) {
    yearlyMap[fy].orders++;

    const wonValue = parseFloat(
      row.orderWonValueInCrWithoutGST ||
      row.estimatedValueInCrWithoutGST ||
      0
    );

    if (!isNaN(wonValue)) {
      yearlyMap[fy].value += wonValue;
    }
  }

  if (status.includes("lost")) {
    yearlyMap[fy].lost++;
  }
});

```

```

// BQ
bqs.forEach(row => {
  const date = parseDate(row.dateOfLetterSubmission);
  if (!date || isNaN(date.getTime())) return;

  const fy = getFinancialYear(date);
  ensureYear(fy);

  yearlyMap[fy].bqs++;
});

// LOST FORMS
lostForms.forEach(row => {
  const fy = row.yearWeLost;
  if (!fy) return;

  ensureYear(fy);
  yearlyMap[fy].lost++;
});

const sortedYears = Object.values(yearlyMap).sort((a, b) =>
  a.fy.localeCompare(b.fy)
);

const yearSummaryData = sortedYears.map((row, index) => {
  let growth = "Baseline";

  if (index > 0) {
    const prev = sortedYears[index - 1].value;

    if (prev > 0) {
      const percent = ((row.value - prev) / prev) * 100;
      growth = `${percent >= 0 ? "+" : ""}${percent.toFixed(1)}%`;
    }
  }
});

return {
  fy: row.fy,
  bqs: row.bqs,
  leads: row.leads,

```

```

    orders: row.orders,
    value: Number(row.value.toFixed(2)),
    winRate:
      row.leads > 0
        ? ` ${((row.orders / row.leads) * 100).toFixed(1)}% `
        : "0%",
    lost: row.lost,
    growth
  };
});

const yearlyValueData = yearSummaryData.map(row => ({
  fy: row.fy,
  value: row.value
}));

const winRateTrendData = yearSummaryData.map(row => ({
  fy: row.fy,
  rate: Number(row.winRate.replace("%", ""))
}));

// KPIs
const bestOrderYear = [...yearSummaryData].sort(
  (a, b) => b.orders - a.orders
)[0];

const bestValueYear = [...yearSummaryData].sort(
  (a, b) => b.value - a.value
)[0];

const totalValue = yearSummaryData.reduce(
  (sum, row) => sum + row.value,
  0
);

const peakWinRate = [...winRateTrendData].sort(
  (a, b) => b.rate - a.rate
)[0];

const avgOrders = yearSummaryData.length
  ? (

```

```
    yearSummaryData.reduce((sum, row) => sum + row.orders, 0) /  
    yearSummaryData.length  
  ).toFixed(0)  
  : 0;
```

```
const yearlyKpis = [  
  {  
    label: "Best Year (Orders)",  
    value: bestOrderYear?.fy || "-",  
    delta: ` ${bestOrderYear?.orders || 0} orders`,  
    deltaType: "up"  
  },  
  {  
    label: "Best Year (Value)",  
    value: bestValueYear?.fy || "-",  
    delta: ` ₹${bestValueYear?.value || 0} Cr`,  
    deltaType: "up"  
  },  
  {  
    label: "Total Value",  
    value: ` ₹${totalValue.toFixed(2)} Cr`,  
    delta: "All years",  
    deltaType: "neutral"  
  },  
  {  
    label: "Peak Win Rate",  
    value: ` ${peakWinRate?.rate || 0}%`,  
    delta: peakWinRate?.fy || "-",  
    deltaType: "neutral"  
  },  
  {  
    label: "Avg Orders/Year",  
    value: avgOrders,  
    delta: "Average",  
    deltaType: "up"  
  }  
];
```

```
res.status(200).json({  
  success: true,  
  yearlyValueData,
```

```

    winRateTrendData,
    yearSummaryData,
    yearlyKpis
  });
} catch (error) {
  console.error(error);

  res.status(500).json({
    success: false,
    message: error.message
  });
}
};

```

Router

```
router.get("/yearly-analysis", getYearlyAnalysis);
```

Frontend integration

Replace:

```
import * as mockData from '../mockData';
```

with state:

```

const [yearlyData, setYearlyData] = useState({
  yearlyValueData: [],
  winRateTrendData: [],
  yearSummaryData: [],
  yearlyKpis: []
});

```

Fetch:

```

useEffect(() => {
  fetch(`${ServerIp}/api/yearly-analysis`)
    .then(res => res.json())
    .then(data => {
      if (data.success) {
        setYearlyData(data);
      }
    });
}, []);

```

Replace:

```
<KPIGrid kpis={yearlyKpis} />
```

with:

```
<KPIGrid kpis={yearlyData.yearlyKpis} />
```


Replace:
mockData.yearlyValueData
with:
yearlyData.yearlyValueData
Replace:
mockData.winRateTrendData
with:
yearlyData.winRateTrendData
Replace:
mockData.yearSummaryData
with:
yearlyData.yearSummaryData

One bug I noticed in your frontend

In YearlyAnalysis.js:

```
const [ServerIp, SetServerIp] = useState("");
```

But I don't see anywhere you set it.

So this fetch:

```
fetch(`${ServerIp}/api/yearly-analysis`)
```

becomes:

```
/api/yearly-analysis
```

That only works if frontend + backend are on same origin.

If you're getting CORS / 404, send me how you set ServerIp elsewhere and I'll fix that too.